



UAVLogBook

Docker Installation Guide

Complete setup for development and production

Version 1.2 · June 2025

MySQL 8.0

Database

Auto-schema
Persistent volume

PHP 8.2 + Apache

App Server

API + SPA
Video streaming

Node 20

Builder

React build
One-shot exit

Nginx

Reverse Proxy

SSL + Rate limit
Optional

Table of Contents

1	System Requirements	3
2	Installing Docker & Docker Compose	4
3	Project Structure	6
4	Configuration (.env)	7
5	Starting the Stack — Development	9
6	Starting the Stack — Production	11
7	SSL Certificates (Let's Encrypt)	13
8	First Login & Account Setup	14
9	Container Reference	15
10	Volume Reference	16
11	Makefile Command Reference	17
12	Updating the Application	18
13	Backup & Restore	19
14	Logs & Debugging	20
15	Troubleshooting	21

1 System Requirements

UAVLogBook runs on any machine or server that supports Docker. No PHP, MySQL, or Node.js installation is needed on the host — everything runs inside containers.

Host Machine Requirements

Component	Minimum	Recommended	Notes
OS	Any 64-bit	Linux (Ubuntu)	Windows/macOS use Docker Desktop
RAM	2 GB	4 GB+	MySQL + PHP + Node builder
Disk	10 GB free	50 GB+	For OS, images, uploads, videos
CPU	2 cores	4 cores+	Log parsing is CPU-intensive
Docker Engine	24.0+	Latest	docker.com/get-docker
Docker Compose	v2.20+	Latest	Included with Docker Desktop
make (optional)	Any	Any	For convenience Makefile commands
openssl (optional)	Any	Any	To generate JWT_SECRET
Ports 80/443	For prod	For prod	Must be free for Nginx profile

What Docker installs inside containers

Service	Image	Version	Purpose
Database	mysql	8.0	All flight data, user accounts, telemetry
App Server	php (custom build)	8.2-apache	REST API, log parsing, video handler
Builder	node	20-alpine	Builds React web app (exits after build)
Proxy	nginx	1.25-alpine	SSL termination, rate limiting (optional)

2 Installing Docker & Docker Compose

Linux (Ubuntu / Debian) — Recommended for servers

```
Terminal — Ubuntu/Debian
# Remove old versions if any
$ sudo apt-get remove docker docker-engine docker.io containerd runc

# Install using the official convenience script (simplest method)
$ curl -fsSL https://get.docker.com | sh

# Add your user to the docker group (so you can run without sudo)
$ sudo usermod -aG docker $USER

# Log out and back in, then verify:
$ docker --version
Docker version 26.x.x, build xxxxxxxx

$ docker compose version
Docker Compose version v2.x.x
```

Alternative: Manual install

If you prefer the manual method: `sudo apt-get update && sudo apt-get install ca-certificates curl gnupg`, then add Docker's GPG key and repository, then `sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin`.
Full instructions at docs.docker.com/engine/install/ubuntu

macOS

```
Terminal — macOS
# Option A: Docker Desktop (GUI + CLI, easiest)
# Download from: https://www.docker.com/products/docker-desktop/
# Install the .dmg, open Docker Desktop, wait for it to start.

# Option B: OrbStack (lighter alternative, macOS only)
# Download from: https://orbstack.dev/

# Verify after install:
$ docker --version && docker compose version
```

Windows 10/11

PowerShell / Windows Terminal

Recommended: WSL 2 + Docker Desktop

Step 1: Enable WSL 2

> wsl --install

> wsl --set-default-version 2

Step 2: Download Docker Desktop for Windows

<https://www.docker.com/products/docker-desktop/>

During install: check 'Use WSL 2 instead of Hyper-V'

Step 3: Open Docker Desktop, wait for engine to start

Step 4: Open PowerShell or Windows Terminal and verify:

> docker --version

> docker compose version

All remaining commands in this guide run in PowerShell or WSL terminal

VPS / Cloud Server (DigitalOcean, Hetzner, AWS EC2, etc.)

VPS SSH terminal

Same as Ubuntu – the convenience script works on all Debian-family distros:

\$ curl -fsSL https://get.docker.com | sh

\$ sudo usermod -aG docker \$USER && newgrp docker

Enable Docker to start on boot:

\$ sudo systemctl enable docker

\$ sudo systemctl start docker

Open firewall ports (if using ufw):

\$ sudo ufw allow 80/tcp

\$ sudo ufw allow 443/tcp

\$ sudo ufw reload

3 Project Structure

After extracting UAVLogBook_v1.2.zip you will have this layout. The `docker/` directory contains all container configuration. You only need to edit `.env` — everything else is ready to run.

<code>uavlogbook_v3/</code>	Project root — run all commands from here
■■■ <code>docker-compose.yml</code>	Main stack: MySQL + PHP + builder + Nginx
■■■ <code>docker-compose.dev.yml</code>	Dev overrides: Xdebug, Vite HMR, MySQL exposed
■■■ <code>.env.example</code>	Environment template — COPY to <code>.env</code> and edit
■■■ <code>.env</code>	Your secrets — created from <code>.env.example</code> (not in git)
■■■ <code>Makefile</code>	Convenience commands: <code>make dev / up / backup</code> etc.
■■■ <code>.dockerignore</code>	Keeps Docker build context fast and secure
■■■ <code>.gitignore</code>	Excludes secrets and runtime data from git
■	
■■■ <code>docker/</code>	All container configuration
■ ■■■ <code>php/</code>	
■ ■■■■ <code>Dockerfile</code>	Multi-stage build: <code>base</code> → <code>dev</code> or <code>production</code>
■ ■■■■ <code>apache.conf</code>	Apache VirtualHost — API routing + video streaming
■ ■■■■ <code>php.ini</code>	PHP limits (8GB uploads, 600s timeout, opcache)
■ ■■■■ <code>nginx/</code>	
■ ■■■■ <code>nginx.conf</code>	Rate limiting zones, compression
■ ■■■■ <code>default.conf</code>	HTTPS server block, SSL, proxy rules
■ ■■■■ <code>proxy_params.conf</code>	Shared proxy headers included by all location blocks
■ ■■■■ <code>certbot.sh</code>	Let's Encrypt certificate helper script
■ ■■■■ <code>mysql-init/</code>	
■ ■■■■ <code>01_init.sql</code>	Auto-runs <code>schema.sql</code> on first container start
■ ■■■■ <code>schema.sql</code>	Full database schema (copy of <code>server/db/schema.sql</code>)
■	
■■■ <code>server/</code>	PHP backend (bind-mounted into app container)
■■■ <code>web/</code>	React web app (built by builder container)

4 Configuration (.env)

All configuration is done in a single `.env` file in the project root. Copy the template and edit it before starting the stack.

```
Terminal
# Copy the template
$ cp .env.example .env

# Edit it (choose your preferred editor)
$ nano .env
$ vim .env
$ code .env    # VS Code
```

Required Settings — must change before first run

Variable	Example value	Description
MYSQL_ROOT_PASSWORD	MyR00tP@ss2025!	MySQL internal root password. Use any strong password.
MYSQL_PASSWORD	MyAppP@ss2025!	Password for the app DB user. Different from root.
JWT_SECRET	a8f3d2c1... (64 hex chars)	Signs all login tokens. Generate: <code>openssl rand -hex 32</code>

```
Generating JWT_SECRET
# Generate a strong JWT secret with openssl:
$ openssl rand -hex 32
a8f3d2c1b4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1

# Paste that value into .env:
JWT_SECRET=a8f3d2c1b4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1
```

All Settings Reference

Variable	Default	When to change
MYSQL_ROOT_PASSWORD	(required)	Always — set a strong password
MYSQL_DATABASE	uavlogbook	Optional — rename the database
MYSQL_USER	uavlogbook_user	Optional — rename the DB user
MYSQL_PASSWORD	(required)	Always — set a strong password
JWT_SECRET	(required)	Always — generate with openssl
ANTHROPIC_API_KEY	(empty = AI disabled)	Optional — for AI log format detection
VITE_API_URL	/api/v1	Leave as-is for Docker (proxy handles routing)

ALLOWED_ORIGINS	http://localhost,http://localhost:8080	Set to your domain in production
NGINX_SERVER_NAME	localhost	Set to your domain when using Nginx profile
PHP_MEMORY_LIMIT	256M	Increase if parsing very large log files
PHP_UPLOAD_MAX_FILESIZE	8192M	Matches 8GB video upload limit
PHP_POST_MAX_SIZE	8192M	Must be >= PHP_UPLOAD_MAX_FILESIZE
PHP_MAX_EXECUTION_TIME	600	Seconds allowed for parsing (10 minutes)
APP_PORT	8080	Host port to access app without Nginx

5 Starting the Stack — Development

Development mode gives you live PHP editing (no rebuild needed), Vite HMR for the React web app, Xdebug for step-debugging PHP, and MySQL exposed on port 3306 for database tools.

1

Extract the zip and enter the project directory

```
$ unzip UAVLogBook_v1.2.zip
$ cd uavlogbook_v3
```

2

Copy and edit the environment file

Open `.env` in any text editor. Set the three required values: `MYSQL_ROOT_PASSWORD`, `MYSQL_PASSWORD`, and `JWT_SECRET`.

```
$ cp .env.example .env
$ nano .env

# Minimum changes required:
MYSQL_ROOT_PASSWORD=YourStrongRootPassword
MYSQL_PASSWORD=YourStrongAppPassword
JWT_SECRET=paste-64-random-chars-here
```

3

Start the development stack

This builds the PHP Docker image, starts MySQL, and launches the Vite dev server. First run takes 3–5 minutes (downloads images, builds PHP image with ffmpeg). Subsequent starts take ~10 seconds.

```
# Using make (if available):
$ make dev

# Or directly with docker compose:
$ docker compose -f docker-compose.yml -f docker-compose.dev.yml up
```

4

Wait for all services to be ready

Watch the logs. You will see:

```
uavlogbook_db      | ready for connections      <- MySQL ready
uavlogbook_builder | Build complete.            <- React app built
uavlogbook_app     | AH00094: Command line:...  <- Apache started
uavlogbook_webdev  | VITE ready in 1234ms       <- Vite HMR ready
```

5

Open the application

```
React app (Vite HMR):  http://localhost:3000
API direct:           http://localhost:8080/api/v1
MySQL (DB tools):     localhost:3306
```

Development features

PHP hot-edit — Edit any file in server/ and refresh the browser — changes reflect instantly. No container rebuild needed.

Vite HMR — Edit web/src/ files and the browser updates in milliseconds without losing state.

Xdebug — Attach your IDE (VS Code, PhpStorm) to localhost:9003. The PHP container listens automatically.

MySQL exposed — Connect TablePlus, DBeaver, or mysql CLI to localhost:3306 with MYSQL_USER/MYSQL_PASSWORD from .env.

Verbose PHP errors — PHP errors shown in the browser response (PHP_DISPLAY_ERRORS=On in dev mode).

Stop the stack

Press Ctrl+C in the terminal to stop. Or in another terminal: make down (or docker compose down). Data in volumes is preserved — the database and uploads survive.

6 Starting the Stack — Production

Option A — Without Nginx (behind existing proxy or for testing)

Use this if you already have a reverse proxy (Cloudflare, Traefik, Caddy) in front, or just want to test production mode on port 8080.

```
Terminal
# Start all services except Nginx
$ make up
# or:
$ docker compose up -d

# Check everything started correctly
$ docker compose ps

# Follow logs
$ make logs
# or:
$ docker compose logs -f

# App is available at:
http://localhost:8080
```

Option B — With Nginx (recommended for internet-facing servers)

Nginx handles SSL termination, HTTP→HTTPS redirect, rate limiting, and caching. Use this for any server that is publicly accessible.

1

Point your domain DNS to the server IP

Log into your domain registrar and set an A record:

yourdomain.com → your.server.ip.address

DNS propagation takes 1–60 minutes depending on registrar.

2

Edit .env — set your domain and production CORS

```
NGINX_SERVER_NAME=yourdomain.com
ALLOWED_ORIGINS=https://yourdomain.com
VITE_API_URL=/api/v1
```

3

Start with the production profile

The production profile adds the Nginx container.

```
$ make prod
# or:
$ docker compose --profile production up -d
```

4

Get a free SSL certificate

DNS must be propagated before running this. certbot.sh uses the Let's Encrypt webroot challenge.

```
$ make ssl D=yourdomain.com E=admin@yourdomain.com
# or:
$ bash docker/nginx/certbot.sh yourdomain.com admin@yourdomain.com
```

5

Verify

<code>https://yourdomain.com</code>	<code><- Web app (HTTPS)</code>
<code>https://yourdomain.com/api/v1</code>	<code><- API</code>
<code>http://yourdomain.com</code>	<code><- Redirects to HTTPS</code>

Startup sequence (automatic)

On first docker compose up: (1) MySQL starts and imports schema.sql automatically. (2) The builder container runs `npm ci && npm run build`, then exits. (3) The app container waits for MySQL health check to pass, then Apache starts. (4) Nginx starts last and waits for the app health check. Subsequent restarts skip the build step and are ready in under 10 seconds.

7 SSL Certificates (Let's Encrypt)

UAVLogBook includes a certbot.sh helper that provisions a free Let's Encrypt certificate with one command. Certificates are valid for 90 days and can be renewed by running the same command again.

Prerequisites

- ✓ Your domain's A record points to this server's public IP
- ✓ The Nginx profile is running (make prod) and port 80 is accessible from the internet
- ✓ The server firewall allows inbound TCP on ports 80 and 443

Get certificate

Terminal

```
$ make ssl D=yourdomain.com E=admin@yourdomain.com
```

```
# What this does behind the scenes:
# 1. Runs certbot in a temporary Docker container
# 2. Uses webroot challenge via /.well-known/acme-challenge/
# 3. Copies fullchain.pem and privkey.pem into the ssl_certs volume
# 4. Reloads Nginx to pick up the new certificate

# On success you will see:
# Congratulations! Your certificate and chain have been saved.
# SSL configured for yourdomain.com
```

Automatic renewal

Add a cron job on the host to renew every 60 days (certificates expire after 90 days):

Cron setup

```
# Open crontab
```

```
$ crontab -e
```

```
# Add this line (runs at 3am on the 1st of every other month):
```

```
0 3 1 */2 * cd /path/to/uavlogbook_v3 && bash docker/nginx/certbot.sh yourdomain.com admin@yourdomain.com
```

Bring-your-own certificate

If you already have a certificate (from a CA, Cloudflare, or wildcard cert), copy fullchain.pem and privkey.pem into the uavlogbook_ssl_certs Docker volume: `docker run --rm -v uavlogbook_ssl_certs:/ssl -v /path/to/your/certs:/src alpine sh -c 'cp /src/fullchain.pem /ssl/ && cp /src/privkey.pem /ssl/'` Then: `docker compose exec nginx nginx -s reload`

8 First Login & Account Setup

1

Open the web app

Development: <http://localhost:3000> or <http://localhost:8080>

Production: <https://yourdomain.com>

2

Register your first account

Click Register, enter your email, choose a strong password (min 8 chars), and set a display name. The first account registered becomes admin.

3

Import your first flight log

Click Import Log in the sidebar. Drop any supported flight log file (.bin, .tlog, .ulg, .skylog, .csv, .gpx, etc.). The AI engine detects the format automatically.

4

Add an FPV video (optional)

Open a flight → FPV Video module → drop an MP4/MOV file. Auto time-sync reads the creation timestamp from the video's MP4 metadata.

API health check

Before logging in, verify the API is responding by visiting:

<http://localhost:8080/api/v1/auth/login> in your browser. You should see: `{"error":"Invalid credentials"}` — that means the API is up and MySQL is connected.

9 Container Reference

uavlogbook_db

Image	mysql:8.0
Ports	Internal only
Internal port	3306
Dev host port	3306 (exposed in dev compose)
Network	uavlogbook_internal only
Restart	unless-stopped
Health check	mysqladmin ping every 10s

MySQL 8.0 database. Starts first. Imports schema.sql on first run via /docker-entrypoint-initdb.d/. Tuned for batch telemetry inserts with large innodb_buffer_pool and max_allowed_packet=256M.

uavlogbook_builder

Image	node:20-alpine
Ports	No network
Run mode	One-shot (exits after build)
Network	None
Output	web_dist volume → /build/dist
Skip condition	dist/index.html exists

Runs npm ci && npm run build once, writes output to the web_dist volume, then exits with code 0. On subsequent docker compose up it detects that dist/index.html already exists and skips the build immediately.

uavlogbook_app

Image	custom (Dockerfile)
Ports	:8080 → :80
Host port	\${APP_PORT:-8080}
Networks	internal + external
Source mount	./server → /var/www/html/api (dev: read-write)
Build target	production (or development in dev compose)
Health check	POST /api/v1/auth/login every 30s

PHP 8.2 + Apache. Serves: REST API at /api/v1/, built React SPA from web_dist volume, video files from uploads_videos volume. Waits for db health check before starting. PHP extensions: pdo_mysql, mbstring, zip, gd, curl, opcache, bcmath. FFmpeg installed for video thumbnails.

uavlogbook_nginx

Image	nginx:1.25-alpine
-------	-------------------

Ports	:80, :443
Ports	80 (HTTP redirect) + 443 (HTTPS)
Profile	production only
SSL certs	ssl_certs volume: /etc/nginx/ssl/
Rate limit zones	auth: 5/min · upload: 10/min · api: 60/min

Optional reverse proxy started with --profile production. Handles HTTP→HTTPS redirect, SSL termination (TLS 1.2/1.3), rate limiting (5 req/min on auth, 10 req/min on uploads, 60 req/min general), HSTS headers, and Accept-Ranges for video seeking.

10 Volume Reference

All persistent data lives in named Docker volumes. They survive docker compose down and are only deleted by make clean or docker volume rm.

Volume name	Mounted at	Contents
uavlogbook_db_data	/var/lib/mysql	MySQL database files. All flights, users, telemetry.
uavlogbook_uploads_logs	/var/www/uploads/logs	Raw log files (.bin, .tlog, .skylog etc.) uploaded by users.
uavlogbook_uploads_videos	/var/www/uploads/videos	FPV video files. Served directly for HTML5 video streaming.
uavlogbook_web_dist	/var/www/html	Compiled React app. Written by builder, read by app container.
uavlogbook_ssl_certs	/etc/nginx/ssl	SSL certificates (fullchain.pem + privkey.pem). Production only.
uavlogbook_letsencrypt	/var/www/certbot	Let's Encrypt webroot for ACME http-01 challenge. Production only.

Inspecting volumes

Terminal

```
# List all UAVLogBook volumes
```

```
$ docker volume ls | grep uavlogbook
```

```
# Inspect a volume (see mount point on host)
```

```
$ docker volume inspect uavlogbook_db_data
```

```
# Browse volume contents (temporary alpine container)
```

```
$ docker run --rm -it -v uavlogbook_uploads_logs:/data alpine ls -la /data
```

```
# Check disk usage
```

```
$ docker system df -v | grep uavlogbook
```

11 Makefile Command Reference

The Makefile provides short aliases for common docker compose commands. If make is not available, the equivalent docker compose command is shown.

DEVELOPMENT

<code>make dev</code>	Start dev stack (foreground, shows logs, Ctrl+C to stop)
<code>make dev-d</code>	Start dev stack in background
<code>make down</code>	Stop and remove all containers (volumes kept)
<code>make ps</code>	Show running containers and their status
<code>make logs</code>	Tail logs from all containers
<code>make logs-app</code>	Tail logs from PHP/Apache only
<code>make shell</code>	Open bash shell inside the PHP container
<code>make db-shell</code>	Open MySQL shell in the database container

PRODUCTION

<code>make up</code>	Start production stack (no Nginx) on port 8080
<code>make prod</code>	Start full production stack including Nginx on 80/443
<code>make ssl D=domain.com E=email</code>	Get/renew Let's Encrypt SSL certificate

MAINTENANCE

<code>make rebuild-web</code>	Force rebuild of React app (use after changing web/src/)
<code>make rebuild-app</code>	Rebuild PHP Docker image (use after changing Dockerfile)
<code>make backup</code>	Dump database to ./backups/uavlogbook_YYYYMMDD_HHMMSS.sql
<code>make restore F=file.sql</code>	Restore database from a backup file
<code>make clean</code>	DESTRUCTIVE: removes all containers AND volumes. Asks to confirm.

12 Updating the Application

Update PHP server code (no image rebuild needed)

In production, server/ is copied into the Docker image at build time. To deploy updated PHP code:

```
Updating PHP code
# Stop the app container
$ docker compose stop app

# Rebuild the app image with the new code
$ docker compose build app

# Restart
$ docker compose up -d app

# Shortcut:
$ make rebuild-app
```

Update React web app (after changing web/src/)

```
Updating the web app
# Force a fresh build of the React app
$ make rebuild-web

# What this does:
# 1. Removes the uavlogbook_web_dist volume
# 2. Starts the builder container → runs npm ci && npm run build
# 3. Restarts the app container to serve the new files
```

Update Docker base images (security patches)

```
Updating base images
# Pull latest versions of all base images
$ docker compose pull

# Rebuild app image with the new base
$ docker compose build --no-cache app

# Restart everything
$ docker compose up -d

# Clean up old image layers
$ docker image prune -f
```

13 Backup & Restore

Database backup

```
Database backup
# One command – saves to ./backups/ with timestamp

$ make backup

# Manual equivalent:

$ mkdir -p backups
$ docker compose exec -T db mysqldump \
    -u$(grep MYSQL_USER .env | cut -d= -f2) \
    -p$(grep MYSQL_PASSWORD .env | cut -d= -f2) \
    $(grep MYSQL_DATABASE .env | cut -d= -f2) \
    > backups/uavlogbook_$(date +%Y%m%d_%H%M%S).sql

# Result: backups/uavlogbook_20250601_143022.sql
```

Database restore

```
Database restore
$ make restore F=backups/uavlogbook_20250601_143022.sql

# Manual equivalent:

$ docker compose exec -T db mysql \
    -u$(grep MYSQL_USER .env | cut -d= -f2) \
    -p$(grep MYSQL_PASSWORD .env | cut -d= -f2) \
    $(grep MYSQL_DATABASE .env | cut -d= -f2) \
    < backups/uavlogbook_20250601_143022.sql
```

Backup uploads and videos

Log files and videos are stored in Docker volumes. Copy them out with a temporary alpine container:

Backing up upload volumes

Backup log files

```
$ docker run --rm \
  -v uavlogbook_uploads_logs:/data \
  -v $(pwd)/backups:/backup \
  alpine tar czf /backup/logs_$(date +%Y%m%d).tar.gz -C /data .
```

Backup video files

```
$ docker run --rm \
  -v uavlogbook_uploads_videos:/data \
  -v $(pwd)/backups:/backup \
  alpine tar czf /backup/videos_$(date +%Y%m%d).tar.gz -C /data .
```

Automated daily backup (cron)

Cron automation

```
$ crontab -e
```

Add – runs at 2am every day:

```
0 2 * * * cd /path/to/uavlogbook_v3 && make backup >> /var/log/uavlogbook-backup.log 2>&1
```

Keep backups for 30 days:

```
30 2 * * * find /path/to/uavlogbook_v3/backups -mtime +30 -delete
```

14 Logs & Debugging

Viewing logs

```
Log commands
# All containers
$ make logs
$ docker compose logs -f

# Single container
$ docker compose logs -f app      # PHP / Apache
$ docker compose logs -f db      # MySQL
$ docker compose logs -f nginx   # Nginx (production only)
$ docker compose logs -f builder # Node build output

# Last 100 lines
$ docker compose logs --tail=100 app
```

Shell access

```
Container shells
# Bash shell in PHP/Apache container
$ make shell
$ docker compose exec app bash

# Inside the container you can:
$ php -v          # check PHP version
$ php -m          # list loaded extensions
$ cat /var/log/apache2/error.log # Apache error log
$ cat /var/log/apache2/php_errors.log # PHP error log
$ ffmpeg -version # check FFmpeg
$ curl localhost/api/v1/auth/login -X POST -H 'Content-Type: application/json' -d '{"email":"x","password":"x"}'

# MySQL shell
$ make db-shell
$ docker compose exec db mysql -uuavlogbook_user -p uavlogbook
mysql> SHOW TABLES;
mysql> SELECT id,email,created_at FROM users;
mysql> SELECT id,parse_status,log_format FROM flights ORDER BY id DESC LIMIT 5;
```

PHP Xdebug (development only)

In dev mode (make dev), Xdebug listens on port 9003. Configure your IDE:

```
Xdebug IDE configuration
# VS Code — add to .vscode/launch.json:
{
  "version": "0.2.0",
  "configurations": [{
    "name": "UAVLogBook Docker PHP",
    "type": "php",
    "request": "launch",
    "port": 9003,
    "pathMappings": {
      "/var/www/html/api": "${workspaceFolder}/server"
    }
  }]
}

# PhpStorm: Settings → PHP → Debug → Xdebug port: 9003
# Add path mapping: /var/www/html/api → /local/path/to/server
```


15 Troubleshooting

docker compose: command not found

Docker Compose v2 uses "docker compose" (space, not hyphen). If you have v1 installed, either upgrade or use "docker-compose" instead. Install Compose v2: `sudo apt-get install docker-compose-plugin`

```
$ docker compose version # should show v2.x.x
```

Permission denied running docker

Your user is not in the docker group. Run: `sudo usermod -aG docker $USER` then log out and back in. Until then, prefix commands with `sudo`.

```
$ sudo docker compose up -d
```

app container exits immediately or keeps restarting

Usually means the `.env` file is missing or has wrong values. Check:

```
$ docker compose logs app
$ docker compose logs db
# Common causes:
# - .env not created (run: cp .env.example .env)
# - MYSQL_PASSWORD wrong (must match in db and app)
# - Port 8080 already in use (change APP_PORT in .env)
```

MySQL health check failing / app waits forever

The db container is not ready yet. On slow machines or first run this can take 60+ seconds. Watch with `docker compose logs db` — wait for "ready for connections".

```
$ docker compose logs -f db
# When you see: ready for connections
# The app will start automatically
```

Port 8080 (or 80/443) already in use

Another service is using the port. Either stop it or change `APP_PORT` in `.env`.

```
# Find what is using port 8080:
$ sudo lsof -i :8080
# Or change the port in .env:
APP_PORT=8090
$ docker compose up -d
```

React build fails in builder container

Usually a Node dependency issue. Rebuild the `web_dist` volume from scratch.

```
$ docker compose stop builder
$ docker volume rm uavlogbook_web_dist
$ docker compose up builder
$ docker compose logs -f builder    # watch for errors
```

API returns 500 Internal Server Error

Check the PHP error log inside the container.

```
$ docker compose exec app cat /var/log/apache2/php_errors.log
$ docker compose exec app cat /var/log/apache2/uavlogbook_error.log
# Common cause: DB_HOST wrong (should be "db" in Docker, not "localhost")
```

Video upload fails / times out

PHP limits may be too low for very large videos. Check docker compose logs app. Increase in .env:
PHP_UPLOAD_MAX_FILESIZE=8192M, PHP_MAX_EXECUTION_TIME=600.

```
# Restart app after changing .env:
$ docker compose up -d app
```

make: command not found

make is not installed. Use the full docker compose commands shown in each make target, or install: `sudo apt-get install make` (Linux) / `xcode-select --install` (macOS)

```
# Instead of: make dev
# Run:          docker compose -f docker-compose.yml -f docker-compose.dev.yml up
```

Nginx SSL: certificate not found

The ssl_certs volume is empty. Run certbot.sh first, or copy your own certs in. Nginx will fail to start if the cert files do not exist.

```
$ make ssl D=yourdomain.com E=admin@yourdomain.com
# Wait for: SSL configured for yourdomain.com
$ docker compose restart nginx
```